

Como proteger seu aplicativo no GitHub

Um guia para quem contrata programadores

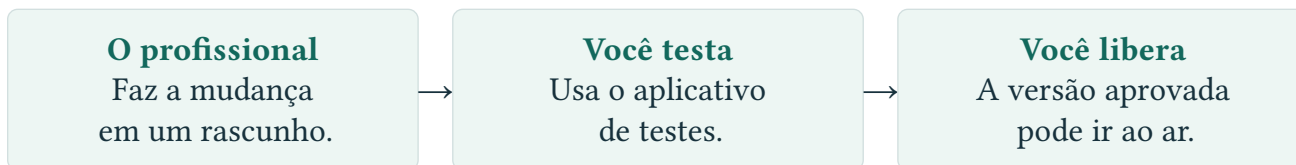
A regra principal

O profissional faz a mudança. Você testa em uma versão do aplicativo ligada ao servidor de testes. Só depois aceita o código e autoriza a publicação para os clientes.

Primeiro: o que é o GitHub?

É um serviço que guarda o código do aplicativo e do servidor, junto com o histórico das mudanças. Cada conjunto de arquivos é chamado de **repositório**. Ele deve ficar na conta da empresa e ser **privado**.

Como o trabalho deve acontecer



Quatro coisas que precisam ficar com você

- ☐ O controle das contas, dos pagamentos e da recuperação de senha.
- ☐ A decisão de aceitar mudanças na versão principal.
- ☐ A decisão de publicar a nova versão para os clientes.
- ☐ Cópias de segurança que o contratado não consiga apagar.

Você não precisa saber programar

O **aplicativo** é o que a pessoa instala. O **servidor** é a parte que recebe os pedidos do aplicativo e cuida dos dados. Faça a configuração com um técnico de confiança; no dia a dia, você testa o aplicativo e decide se aprova.

Isso evita perder tudo?

Reduz muito o risco. Proteger o código e separar os testes ajuda a evitar problemas. Para recuperar o aplicativo, o servidor e os dados, mantenha também cópias de segurança fora do alcance do contratado.

As imagens são de documentações oficiais. Os botões podem aparecer em inglês; as legendas explicam o que procurar. O exemplo considera um app para celular.

1. A empresa precisa ter as chaves

Faça uma vez, antes de liberar o acesso ao profissional.

1. Use uma **organização**: é o espaço da empresa no GitHub. Você deve ser o proprietário, com seu próprio login. Se o projeto estiver na conta do contratado, peça uma cópia de segurança e a transferência para a empresa.
2. Guarde com você a hospedagem, as cópias de segurança e as contas de publicação: **Google Play Console** e/ou **Apple Developer** / **App Store Connect**. A hospedagem mantém o servidor funcionando. As contas das lojas devem pertencer à empresa.
3. Ative o login em duas etapas, com uma confirmação além da senha, e guarde os códigos de recuperação. Exija isso dos participantes também. [13]

Dê acesso para trabalhar, sem entregar a conta

Em cada repositório, abra **Settings** (configurações) e a área de pessoas com acesso. Use **Add people** para convidar o profissional. Escolha **Write**, que permite trabalhar no código. Reserve **Admin** e **Owner**, que dão controle amplo, para você. [1]

Who has access

PRIVATE REPOSITORY

Only those with access to this repository can view it.

[Manage](#)

BASE ROLE
None

No base role set. Only **Owners** and those with direct access can access this repository.

[Set base role](#)

DIRECT ACCESS

8 have access to this repository.
1 member. 2 outside collaborators. 5 teams.

Manage access

[Create team](#) [Add people](#) [Add teams](#)

☐ Select all
Type ▾ Role ▾

Find a team, organization member or outside collaborator...

☐
octo-team
@octo-org/octo-team • 1 member

Role: Read ▾ Remove

☐
The Octocat
octocat

Mixed roles

Role: Write ▾ Remove

Add people adiciona uma pessoa. Role mostra o nível de acesso. Para o contratado, escolha Write. Os nomes da imagem são exemplos. Imagem: GitHub.

Confira com quem configura

Os dois repositórios estão privados. O acesso padrão da organização está em **None** (nenhum). O contratado não recebe acesso maior por outra equipe. Cada pessoa usa sua própria conta. [12]

Se você escolher outro administrador de confiança, ele também terá poder para mudar as proteções. Se quiser que só você tenha esse poder, mantenha apenas você como administrador e proprietário.

2. Organize o aplicativo e o servidor

Dois repositórios são uma opção, não uma exigência. Neste guia, vamos usar um chamado **aplicativo** e outro chamado **servidor**. Se o código já fica junto, os mesmos cuidados podem ser aplicados sem separá-lo.

Cada repositório terá duas **branches**: linhas de trabalho do código. A **main** guarda o que você aprovou. A **staging** recebe o que será testado.

Repositório	main	staging
aplicativo	Código aprovado do app	Código do app de testes
servidor	Código aprovado do servidor	Código do servidor de testes

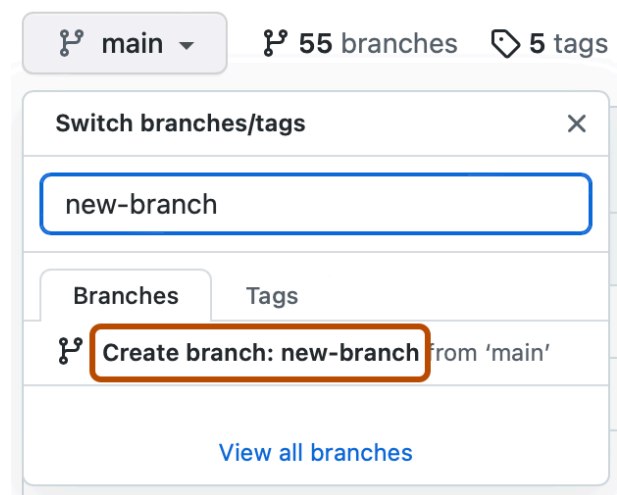
Prepare as branches nos dois repositórios

Faça uma cópia de segurança. Peça ao técnico para colocar o código aprovado em **main**, preservando o histórico. Em **Settings** → **General**, escolha **main** como padrão (**Default branch**). Repita nos dois repositórios. [5]

Crie staging a partir da main

1. Abra a página do projeto e selecione **main** no botão de branches.
2. Abra o mesmo botão e digite **staging** no campo de busca.
3. Clique em **Create branch** para criar a versão de testes. [4]

Na imagem, onde aparece **new-branch**, você deve digitar **staging**.



O novo trabalho parte da **main**. A imagem usa um nome de exemplo. Imagem: GitHub.

Repositório e ambiente são coisas diferentes

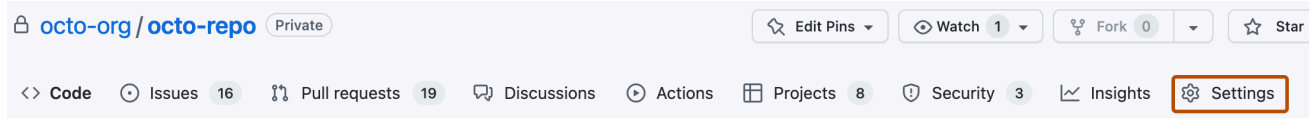
Os repositórios separam o código do app e do servidor. Os ambientes separam os testes dos clientes. Não é preciso copiar todo o projeto para um repositório de “testes” e outro de “produção”.

O profissional trabalha primeiro em um rascunho

Ele cria um rascunho e pede para levar a mudança à **staging**. Esse pedido se chama **pull request**, ou **PR**. Depois de testar, você aceita o pedido de **staging** para **main** em cada repositório que mudou.

3. Deixe só você aceitar mudanças

Esta página é para configurar junto com o responsável técnico. Para um projeto privado dentro de uma organização, use o plano **GitHub Team** ou **Enterprise**. O plano gratuito da organização não aplica as proteções descritas aqui. [2]



Na página do projeto, Settings abre as configurações. Imagem: GitHub.

Repita nos dois repositórios: abra **Settings** → **Branches** e **Add classic branch protection rule**. No campo **Branch name pattern**, escreva **main**. Configure e salve a regra abaixo. [3]

O que pedir ao responsável técnico	Como deixar
Toda mudança precisa passar por um pedido (PR).	Ligado
O pedido precisa de pelo menos uma aprovação.	1 aprovação
Mudou o código? A aprovação anterior perde a validade.	Ligado
Os testes automáticos precisam passar.	Ligado; selecionar os testes reais
O pedido deve incluir a main atual e resolver as pendências da revisão.	Ligado
Limitar quem pode alterar a main: Restrict who can push to matching branches .	Só a sua conta
Aplicar as regras também aos administradores: Do not allow bypassing the above settings .	Ligado
Permitir pular o pedido de mudança.	Ninguém
Permitir descartar uma revisão.	Só você
Permitir apagar a branch ou reescrever o histórico à força.	Ambos desligados

O responsável deve criar e executar os testes antes de selecioná-los como obrigatórios. Peça também para desligar a aceitação automática de pedidos (**auto-merge**) e conferir se há outras regras interferindo. [3]

Apenas exigir uma aprovação não basta

A restrição de quem altera a main é o que reserva a decisão final para você. Não coloque o contratado nem programas automáticos nessa lista. Administradores continuam tendo poder para mudar as regras. [2]

Na **staging**, também exija pedidos e testes e impeça apagar a branch ou forçar mudanças no histórico. O contratado pode aceitar pedidos nela. Faça um teste com a conta dele: deve conseguir trabalhar na **staging**, mas não aceitar pedidos na **main**, mesmo com sua aprovação.

4. Separe os testes dos clientes

Sim, este é um fluxo comum para aplicativos. A documentação do Flutter mostra versões de testes e de produção, inclusive com endereços de servidor diferentes. É um exemplo dessa prática; seu app não precisa usar Flutter. [14]

VOCÊ TESTA

App de testes



Servidor de testes — staging



Dados fictícios e contas de teste

CLIENTES USAM

App público



Servidor de produção



Dados reais dos clientes

O que pedir ao responsável técnico

1. **Configure o endereço dentro de cada versão do app.** O app de testes só conversa com staging. O app público só conversa com produção. Nenhuma das versões troca de ambiente sozinha quando há erro.
2. **Separe os serviços e os dados.** Use servidores ou projetos isolados, contas e senhas diferentes. Staging não pode acessar dados nem backups de produção. Se usar Firebase, crie projetos separados por ambiente. [20]
3. **Separe pagamentos e mensagens.** Use cobranças fictícias. E-mails e notificações dos testes não devem chegar aos clientes.
4. **Faça o código certo chegar ao lugar certo.** A staging de cada repositório alimenta os testes. A main fornece o código aprovado para produção. Aceitar um pedido no GitHub não deve publicar automaticamente para os clientes.
5. **Guarde o controle da publicação.** Só você libera o servidor e o app público. Revise permissões na hospedagem, nas lojas e nos programas automáticos. Proteja também as chaves e certificados usados para publicar o app.

Teste sem colocar as senhas reais em risco

O app instalado não deve conter senhas de administrador. Os programas que executam código ainda não aprovado não podem receber credenciais de produção. A área geral de “secrets” do GitHub não protege uma senha de quem pode mudar esses programas. [11]

Se a publicação usar GitHub Actions

É o recurso de tarefas automáticas. Em cada repositório, o técnico deve restringir o ambiente production à branch main, guardar nele as credenciais e publicar apenas a versão aprovada. [6]

No GitHub Team, projetos privados não têm a aprovação manual de publicação do próprio GitHub. Use a liberação na hospedagem e nas lojas, com acessos sob seu controle. O técnico deve demonstrar que o contratado não consegue contornar essa decisão. [6]

5. Instale o aplicativo de testes

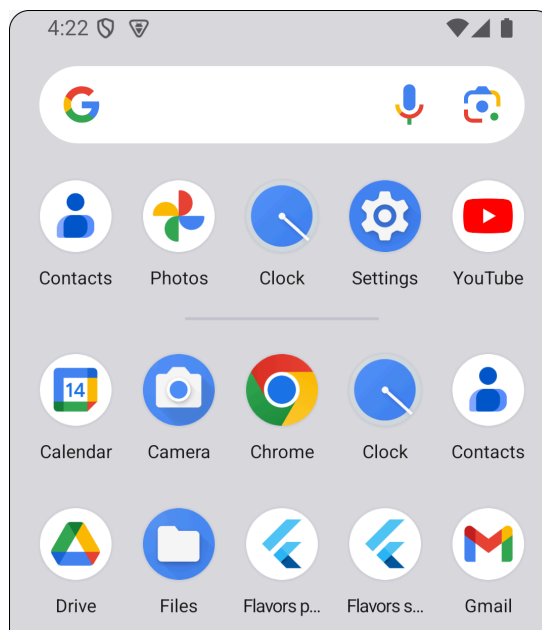
Você deve experimentar a mudança **no aplicativo instalado**, usando o servidor staging. Abrir apenas uma página do servidor não testa a experiência do usuário.

Evite confundir as versões

Peça o nome “**Meu App — TESTE**”, um ícone diferente e a indicação **TESTES** dentro do aplicativo.

Se quiser manter as duas versões instaladas juntas, o técnico deve configurar identificadores diferentes para elas. Isso não exige outro repositório. [15]

A imagem mostra um exemplo oficial com dois aplicativos: produção e staging. Os nomes aparecem abreviados na tela. [14]



Na última linha, Flavors p... e Flavors s... são as duas versões do mesmo exemplo. Imagem: Flutter / Google.

Como receber a versão de testes

- **Se for para iPhone:** o responsável envia um convite pelo **TestFlight**. Você instala o TestFlight, aceita o convite e instala o app de testes. [16]
- **Se for para Android:** o responsável envia um convite de **teste interno ou fechado do Google Play**. Você entra com a conta convidada e segue o link de instalação. [17]

O convite não escolhe o servidor

TestFlight e Google Play distribuem o arquivo do app. É o técnico quem configura sua ligação com staging. Peça uma demonstração: uma conta ou registro criado no teste deve aparecer apenas nos dados de teste.

Deixe a publicação pública nas suas mãos

Na Apple, peça a **liberação manual** da versão na App Store. [18] No Google Play, use **Publicação Gerenciada** quando disponível; ela não cobre o primeiro lançamento nem as atualizações de teste interno. [19]

Esses botões não substituem as permissões. O contratado não deve poder liberar o app público, mudar a regra ou usar uma chave de publicação para contorná-la. Mantenha na empresa o controle das contas e da recuperação.

6. Teste, aprove e publique

Repita para cada entrega. Registre o número do app de testes e a versão do servidor staging que foram usados juntos.

1 Receba a entrega para testar.

Peça a lista do que mudou e instale o app de testes. O profissional prepara um pedido de staging para main em cada repositório alterado. Se só o servidor mudou, teste-o com o app correspondente.

Antes de aprovar, confira

- ☐ A mudança pedida funciona, inclusive quando você erra um campo.
- ☐ Login, busca, cadastro e outras tarefas antigas continuam funcionando.
- ☐ Nos tipos de celular atendidos, funcionam telas, câmera e notificações usadas pelo app.
- ☐ Os dados ficam em staging, sem cobranças reais nem acesso a dados de outra pessoa.

2 Registre sua aprovação no pedido.

Em cada pedido, confira destino **main** e origem **staging**. Abra **Files changed** (arquivos alterados) e **Review changes** (revisar). Anote as versões do app e do servidor. Marque **Approve** (aprovar) e **Submit review** (enviar revisão). [7]

Adding sparkles #45834

Edit <> Code

Open octocat wants to merge 6 commits into main from octocats-branch

Conversation 34 Commits 6 Checks 57

Files changed 2

+32 -4

Changes from all commits File filter Conversations

0 / 2 files viewed

Review in codespace

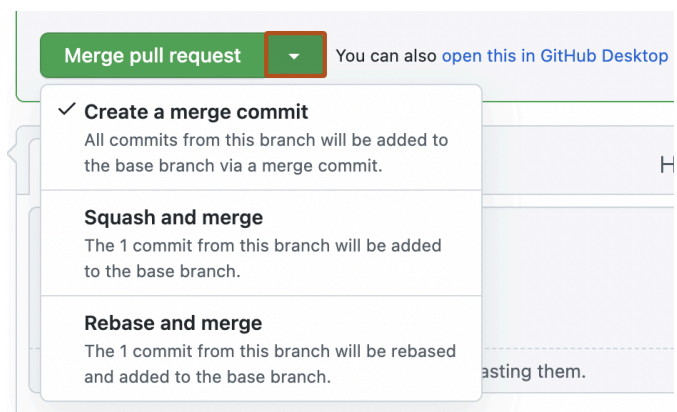
Review changes

Review changes abre a revisão. No seu pedido de entrega, a origem deve ser staging. Imagem: GitHub.

3 Aceite o código aprovado.

Com os testes aprovados, volte à conversa. Clique em **Merge pull request** e em **Confirm merge**. Repita no outro repositório, se ele mudou. [8]

Aceitar o código não libera o servidor nem o aplicativo para os clientes.



Merge aceita a mudança na main. Use a opção Create a merge commit preparada pelo técnico. Imagem: GitHub.

Antes de publicar: o técnico confere o código final aprovado e valida o pacote público do app, configurado para produção. Não envie às lojas o pacote que aponta para staging. Se algo mudou, teste novamente. Em mudanças sensíveis, peça revisão de outro técnico.

Você libera as duas partes: combine a ordem da atualização do servidor e do app nas lojas. O servidor deve atender também o app que já está instalado enquanto a atualização chega. Confira o resultado; depois, o técnico atualiza as branches de testes para a próxima entrega.

7. Guarde uma saída de emergência

Backup quer dizer cópia de segurança. Ela deve permitir recuperar o projeto sem depender do contratado.

Peça para configurar

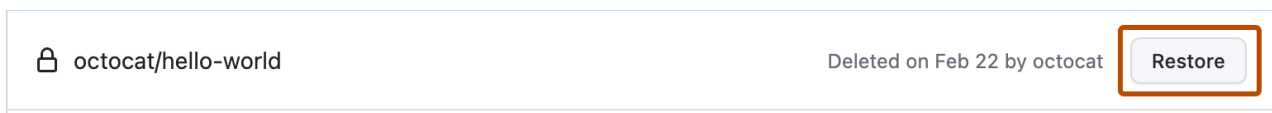
- ☐ **Uma cópia completa dos dois repositórios e dos históricos**, incluindo branches, marcações de versões e arquivos grandes. Baixar só os arquivos atuais não guarda todo o histórico. [9]
- ☐ **Cópias dos dados e arquivos dos clientes**. Guarde também configurações, pacotes publicados do app e meios de recuperar as contas e chaves de publicação.
- ☐ **Um local sob seu controle**, fora da conta de trabalho do contratado. Use cópias protegidas por criptografia e guarde as chaves de recuperação. Nem o contratado nem seus programas devem conseguir apagar as cópias antigas.
- ☐ **Cópias de várias datas**. Comece com cópias diárias e antes de publicar mudanças. Guarde, por exemplo, 90 dias; aumente a frequência se perder um dia de trabalho for inaceitável. Mantenha outra cópia em outro serviço ou desconectada.
- ☐ **Um teste de recuperação agora e a cada três meses**. Peça para restaurar em local separado e mostrar o app funcionando com o servidor e os dados recuperados. Receba avisos se uma cópia falhar.

Não vale como única cópia

Outra branch dentro do mesmo projeto, uma pasta no servidor do contratado ou uma cópia que simplesmente repete tudo o que foi apagado. É preciso conseguir voltar a uma data anterior.

Se o projeto for apagado no GitHub

Peça ajuda imediatamente. Como proprietário, abra as configurações da organização e procure **Deleted repositories** (projetos apagados). Para projetos que atendem às condições do GitHub, há recuperação por até 90 dias. Existem exceções; conte também com suas próprias cópias. [10]



Restore significa restaurar. Use essa opção se o projeto aparecer na lista de apagados. *Imagem: GitHub.*

Se uma atualização der problema

Pause a distribuição. O servidor pode voltar a uma versão anterior, se for seguro para os dados. O app já instalado no celular não volta automaticamente: pode ser necessário publicar uma correção. Recuperar o código também não recupera dados apagados.

Quando o contrato terminar

Remova acessos ao GitHub, à hospedagem e às lojas. Revise chaves e certificados com o técnico e troque senhas compartilhadas. Confira os backups. Remover a pessoa do GitHub não cancela toda chave de acesso nem apaga cópias que ela já baixou. [1]

8. Confira antes de considerar pronto

Peça uma demonstração. Use uma conta de contratado para conferir as restrições. Testes de apagar ou forçar mudanças devem acontecer só em uma cópia descartável.

- ☐ Consigo entrar nas contas e recuperá-las sem ajuda do contratado.
- ☐ Os dois repositórios são privados e o plano aplica as proteções.
- ☐ O contratado trabalha, mas não aceita mudanças na main de nenhum deles.
- ☐ Se um teste falha ou o código muda, a aprovação antiga não basta.
- ☐ O app de testes aponta para staging e não acessa os dados dos clientes.
- ☐ Só eu libero o servidor e o app público; sei quais versões foram testadas juntas.
- ☐ Vi uma cópia ser recuperada e sei quem chamar em caso de problema.

Links: aplicativo e servidor	
Convite para instalar o app de testes	
Onde ficam as cópias	
Contato para recuperação	
Data do teste de recuperação	

Fontes e imagens

Fontes oficiais consultadas em 21/09/2026. Links e créditos são clicáveis. As telas podem mudar. Dois repositórios e branches staging são a organização sugerida neste guia; outras equipes podem organizar o código de outra forma.

1. Quem pode fazer o quê no projeto
2. Proteção de branches e planos necessários
3. Como configurar a proteção
4. Como criar uma branch
5. Como escolher a branch principal
6. Configurações dos ambientes e limites dos planos
7. Como registrar uma aprovação
8. Como aceitar a mudança na versão principal
9. Cópias de segurança do projeto
10. Recuperação de um projeto apagado
11. Segurança dos programas que executam tarefas automáticas
12. Como gerenciar as pessoas com acesso
13. Proteção do login em duas etapas
14. App de testes e app de produção: exemplo oficial
15. Versões diferentes do mesmo app no Android
16. TestFlight: instalar uma versão de testes da Apple
17. Testes internos e fechados no Google Play
18. Liberação manual de uma versão na App Store
19. Controle da publicação no Google Play
20. Separação dos ambientes no Firebase

Capturas: GitHub, Inc. e colaboradores de [github/docs](#) imagem dos dois apps: [Flutter](#) / [Google](#) e [colaboradores](#). Licença CC BY 4.0. Imagens originais, apenas redimensionadas, com legendas acrescentadas. São exemplos públicos, não capturas da sua conta. Os desenhos explicativos foram feitos para este guia.

Este material é um guia. A configuração do projeto, da hospedagem e das cópias deve ser feita e conferida com o responsável técnico.